



GUÍA DE AVANCE N.º 1 – PROYECTO FINAL

PRIMERA ENTREGA SEMANA 5

Fases que comprende esta guía

Semana	Contenido	Fase	Resultado
1-2	Fundamentos de Java	Preparación	Repaso de entorno, estructura, variables, tipos y operadores.
3	Clases, atributos, métodos, modificadores y constructor	Fase 1	Análisis y diseño inicial.
4	Objetos, mensajes, métodos con parámetros/retorno, BufferedReader y try-catch	Fase 2	Primera implementación con objetos, métodos, entrada y validación.

Objetivo

Iniciar el desarrollo progresivo del proyecto final mediante el análisis del problema, la identificación de clases y responsabilidades, y la construcción de una primera solución orientada a objetos utilizando constructores, objetos, métodos, entrada de datos y manejo básico de excepciones.

Indicaciones generales

- Trabajen en grupo utilizando únicamente el enunciado asignado.
- No deben terminar el proyecto en esta etapa; implementen solo los contenidos estudiados hasta la semana 4.
- Todos los integrantes deben participar y comprender el código desarrollado.
- Identificar las clases forma parte del análisis; la guía no les proporciona la estructura de la solución.
- Conserven este avance porque será la base de las siguientes entregas.

En esta etapa NO se exige desarrollar completamente menús, ciclos, arreglos, herencia, polimorfismo ni colecciones. Estos elementos se incorporarán cuando sean estudiados.

PARTE I. Análisis del problema

Lean cuidadosamente el enunciado y respondan antes de programar.

1. ¿Cuál es el propósito principal del programa?

El programa busca resolver la necesidad de comunicación directa entre personas conectadas desde diferentes equipos. Para ello, se desarrollará una aplicación de mensajería en línea que permita seleccionar a otro usuario, enviarle mensajes y recibir respuestas en tiempo real desde una interfaz sencilla.

Además de mensajes de texto, se contempla el envío de imágenes, videos y otros archivos. La aplicación deberá informar si el usuario está conectado, si el mensaje puede enviarse o si ocurrió algún problema durante la comunicación.

2. ¿Quiénes o qué elementos participan en el problema?

Los principales participantes son los usuarios de la aplicación. Durante una conversación, un usuario puede actuar como remitente al enviar información y como destinatario al recibirla.

También intervienen los siguientes elementos:

- La aplicación cliente utilizada por cada usuario.
- El servidor encargado de recibir y dirigir los mensajes.
- La conexión de red que permite la comunicación.
- Los mensajes de texto intercambiados.
- Los archivos adjuntos, como imágenes o videos.
- Las conversaciones que organizan los mensajes entre los usuarios.

El servidor no participa como una persona, pero es necesario para determinar quién está conectado y entregar la información al destinatario correcto.

3. ¿Qué datos debe recibir o conocer el programa?

El programa deberá solicitar o conocer la siguiente información:

- Nombre o identificador del usuario.
- Destinatario seleccionado.
- Contenido del mensaje.
- Archivo que el usuario desea enviar, cuando corresponda.
- Dirección y puerto del servidor al que se conectará la aplicación.

Otros datos podrán ser generados automáticamente por el sistema:

- Fecha y hora del envío.
- Estado de conexión del usuario.
- Tipo, nombre y tamaño del archivo.
- Estado del mensaje o de la transferencia.
- Identificación del remitente.

Antes de enviar la información, el programa deberá comprobar que el mensaje no esté vacío, que el destinatario esté disponible y que el archivo cumpla con los límites establecidos.

4. ¿Qué información deberá mantener cada elemento importante?

Mientras la aplicación se encuentre funcionando, deberá mantener la información necesaria para gestionar la comunicación:

- Del usuario: identificador, nombre, estado de conexión y conversación seleccionada.
- Del mensaje: remitente, destinatario, contenido, fecha, hora y estado del envío.
- Del archivo adjunto: nombre, formato, tamaño, remitente, destinatario y progreso de transferencia.
- De la conversación: participantes y mensajes intercambiados durante la sesión.
- De la conexión: dirección del servidor, puerto y estado de la comunicación.
- Del servidor: usuarios conectados y conexión correspondiente a cada uno.

Como el proyecto no utilizará una base de datos, inicialmente esta información se conservará durante la sesión. Al cerrar el servidor o la aplicación, el historial no se mantendrá de forma permanente. El almacenamiento local podrá evaluarse posteriormente como una mejora, sin formar parte de este avance.

5. ¿Cuáles son los procesos principales que deberá realizar?

El sistema deberá realizar los siguientes procesos:

- Permitir que el usuario escriba un nombre y se conecte al servidor.
- Verificar que el nombre sea válido y no esté siendo utilizado.
- Mostrar cuáles usuarios se encuentran conectados.
- Permitir que el usuario seleccione con quién desea conversar.
- Validar el contenido antes de enviarlo.
- Enviar el mensaje al servidor.
- Identificar al destinatario y dirigir el mensaje.
- Recibir y mostrar los mensajes sin detener el uso de la interfaz.
- Permitir la selección y transferencia de imágenes, videos u otros archivos.
- Mostrar el progreso y resultado de la transferencia.
- Actualizar la lista cuando alguien se conecte o desconecte.
- Manejar problemas como pérdida de conexión, destinatario no disponible o archivo inválido.
- Cerrar la sesión del usuario de forma segura.

6. ¿Qué resultados deberá mostrar al usuario?

La aplicación deberá presentar la información de forma clara y fácil de comprender. Entre los resultados esperados se encuentran:

- Confirmación de conexión con el servidor.
- Nombre del usuario que inició la sesión.
- Lista actualizada de usuarios conectados.
- Conversación con los mensajes enviados y recibidos.
- Nombre del remitente y hora de cada mensaje.
- Estado del mensaje, por ejemplo, enviado o no entregado.
- Vista previa de las imágenes recibidas.
- Nombre, tipo y tamaño de los archivos.
- Progreso de las transferencias.
- Confirmación cuando un archivo sea recibido.
- Avisos cuando un usuario se conecte o desconecte.
- Mensajes de error comprensibles.

PARTE II. Identificación preliminar de clases

Identifiquen elementos del problema que podrían representarse mediante clases. No conviertan automáticamente cada proceso en una clase.

Clase propuesta	¿Qué representa?	Atributos preliminares	Métodos / responsabilidades
Usuario	Una persona que utiliza la aplicación y puede actuar como remitente o destinatario.	int idUsuario; String nombreUsuario; String estadoConexion.	Mostrar sus datos, cambiar su estado de conexión y proporcionar su información básica.

Mensaje	La información de texto que un usuario envía a otro dentro de una conversación.	int idMensaje; Usuario remitente; Usuario destinatario; String contenido; String fechaHora; String estadoEnvio.	Validar y mostrar el mensaje, además de proporcionar un resumen legible.
Conversacion	El espacio lógico que relaciona a dos usuarios y organiza su comunicación.	int idConversacion; Usuario participanteUno; Usuario participanteDos; Mensaje ultimoMensaje.	Iniciar la conversación, registrar el último mensaje y verificar participantes.
SistemaMensajeria	La clase que coordina la prueba local de usuarios, conversación y mensajes.	Usuario usuarioActual; Usuario destinatarioActual; Conversacion conversacionActiva.	Iniciar una sesión local, seleccionar destinatario, crear una conversación y enviar un mensaje de prueba.
ArchivoAdjunto	Un archivo compartido mediante el chat; se diseñará, pero no se implementará en este avance.	String nombreArchivo; String tipoArchivo; double tamanoBytes; Usuario remitente; Usuario destinatario; String estadoTransferencia.	validarArchivo(), actualizarProgreso(), estaCompleto() (funcionalidad futura).
ServidorMensajeria	Componente futuro que administrará conexiones y dirigirá mensajes al usuario correcto.	String direccionServidor; int puerto.	iniciarServidor(), registrarUsuario(), reenviarMensaje() (funcionalidad futura).
Pregunta orientadora: ¿este elemento representa una persona, objeto, concepto o entidad relevante que necesita mantener información y realizar algún comportamiento?			

PARTE III. Diseño inicial

Completen una ficha por cada clase propuesta. Pueden copiar esta tabla si necesitan más.

Alcance, requisitos y arquitectura inicial

Alcance del proyecto final: Wisp será una aplicación cliente-servidor que permitirá conversaciones privadas, mensajes de texto, estado de conexión y, en una etapa posterior, transferencia de archivos. Estos elementos describen la solución completa, no necesariamente lo que se implementará en esta entrega.

Alcance de este avance: se construirá una primera prueba de consola con Programación Orientada a Objetos. Se crearán objetos de Usuario, Mensaje, Conversacion y SistemaMensajeria para registrar datos básicos, formar una conversación y simular localmente el envío de un mensaje. No se

implementarán todavía sockets, transferencia real de archivos, interfaz gráfica, arreglos completos, colecciones ni multihilo.

Restricciones y arquitectura inicial: no se utilizará base de datos; la información solo existirá mientras se ejecute el programa. Main solicitará los datos y coordinará la prueba. SistemaMensajeria organizará la lógica local. Usuario, Conversacion y Mensaje representarán la información principal. ArchivoAdjunto y ServidorMensajeria permanecen como clases de diseño para etapas posteriores, cuando se estudien archivos y comunicación por red.

Nombre de la clase	Usuario
Responsabilidad principal	Representar a una persona que puede participar en conversaciones y mantener su estado de conexión.
Atributos y tipo de dato	int idUsuario; String nombreUsuario; String estadoConexion.
Modificadores de acceso	private para todos los atributos; public para constructor y métodos de uso externo.
Constructor: datos que recibirá	int idUsuario, String nombreUsuario. El estado de conexión inicia como "Desconectado".
Métodos iniciales	mostrarDatos(); cambiarEstadoConexion(String nuevoEstado); obtenerNombreUsuario().

Nombre de la clase	Mensaje
Responsabilidad principal	Representar un mensaje de texto entre un remitente y un destinatario, con su estado de envío.
Atributos y tipo de dato	int idMensaje; Usuario remitente; Usuario destinatario; String contenido; String fechaHora; String estadoEnvio.
Modificadores de acceso	private para todos los atributos; public para constructor y métodos de uso externo.
Constructor: datos que recibirá	int idMensaje, Usuario remitente, Usuario destinatario, String contenido, String fechaHora.
Métodos iniciales	validarContenido(); mostrarMensaje(); obtenerResumen().

Nombre de la clase	Conversacion
Responsabilidad principal	Representar la relación entre dos usuarios y conservar el último mensaje durante esta primera etapa.
Atributos y tipo de dato	int idConversacion; Usuario participanteUno; Usuario participanteDos; Mensaje ultimoMensaje.
Modificadores de acceso	private para todos los atributos; public para constructor y métodos de uso externo.
Constructor: datos que recibirá	int idConversacion, Usuario participanteUno, Usuario participanteDos.
Métodos iniciales	registrarUltimoMensaje(Mensaje mensaje); mostrarResumen(); contieneParticipante(Usuario usuario).

Nombre de la clase	SistemaMensajeria
Responsabilidad principal	Coordinar la prueba local: usuario actual, destinatario, conversación y envío de un mensaje.
Atributos y tipo de dato	Usuario usuarioActual; Usuario destinatarioActual; Conversacion conversacionActiva.
Modificadores de acceso	private para referencias a objetos; public para constructor y operaciones de coordinación.
Constructor: datos que recibirá	No requiere datos iniciales; las referencias se asignan durante la prueba desde Main.
Métodos iniciales	iniciarSesion(Usuario usuario); seleccionarDestinatario(Usuario usuario); crearConversacion(); enviarMensaje(String contenido).

Nombre de la clase	ArchivoAdjunto
Responsabilidad principal	Representar un archivo adjunto como diseño de una entrega futura; no se construirá ni transferirá aún.

Atributos y tipo de dato	String nombreArchivo; String tipoArchivo; double tamanoBytes; Usuario remitente; Usuario destinatario; String estadoTransferencia.
Modificadores de acceso	private para atributos; public para métodos de uso externo.
Constructor: datos que recibirá	String nombreArchivo, String tipoArchivo, double tamanoBytes, Usuario remitente, Usuario destinatario.
Métodos iniciales	validarArchivo(), actualizarProgreso(), estaCompleto() (funcionalidad futura).

Nombre de la clase	ServidorMensajeria
Responsabilidad principal	Representar el componente futuro que recibirá conexiones y reenviará mensajes.
Atributos y tipo de dato	String direccionServidor; int puerto.
Modificadores de acceso	private para atributos; public para las operaciones que se implementarán en la fase de red.
Constructor: datos que recibirá	String direccionServidor, int puerto.
Métodos iniciales	iniciarServidor(), registrarUsuario(Usuario usuario), reenviarMensaje(Mensaje mensaje) (fase futura).

PARTE IV. Construcción de las clases en Java

- Crear en IntelliJ IDEA las clases identificadas.
- Definir atributos con tipos de datos y modificadores de acceso apropiados.
- Crear constructores para inicializar los objetos.
- Implementar los métodos que ya puedan desarrollar.
- Incluir comentarios breves en los procesos importantes.

PARTE V. Creación e instanciación de objetos

Desde Main, creen objetos y utilicen sus métodos. Deben poder explicar la relación:

CLASE → CONSTRUCTOR → new → OBJETO → LLAMADA A MÉTODO

Ejemplo conceptual (no pertenece a ninguno de los proyectos):

```
Producto productol = new Producto("P001", "Cuaderno", 2.50);
productol.mostrarDatos();
```

PARTE VI. Métodos

Identifiquen procesos que correspondan a métodos de sus clases. Utilicen, cuando sea aplicable, métodos sin parámetros, métodos con parámetros y métodos que retornen valores. Los métodos de ArchivoAdjunto y ServidorMensajería se muestran únicamente como diseño futuro; no forman parte de la implementación de este avance.

Método	Clase	¿Recibe parámetros?	¿Retorna valor?	Responsabilidad
mostrarDatos	Usuario	No	No	Mostrar el identificador, nombre de usuario y estado de conexión.
cambiarEstadoConexion	Usuario	Sí, String nuevoEstado	No	Actualizar el estado de conexión del usuario.
obtenerResumen	Mensaje	No	Sí, String	Construir y devolver una representación legible del mensaje.
registrarUltimoMensaje	Conversacion	Sí, Mensaje	Sí, boolean	Guardar el mensaje como último mensaje de la conversación.
enviarMensaje	SistemaMensajería	Sí, String contenido	Sí, boolean	Crear un mensaje con el contenido indicado y registrarlo en la conversación activa.

PARTE VII. Entrada de datos y manejo inicial de errores

Incorporen BufferedReader para capturar parte de la información requerida. Cuando conviertan texto a un tipo numérico, utilicen try-catch para evitar que una entrada incorrecta termine inesperadamente el programa.

Aplicación prevista en Wisp para este avance: Main solicitará el identificador numérico y el nombre del usuario, el destinatario y el contenido del mensaje. El try-catch se aplicará al convertir el identificador a int; si la conversión falla, se mostrará un aviso claro y no se creará el usuario con datos inválidos.

- Solicitar datos relacionados con el proyecto.
- Convertir datos cuando corresponda (por ejemplo, int o double).
- Aplicar try-catch al menos en una entrada susceptible de error de conversión.
- Mostrar un mensaje comprensible cuando el dato no sea válido.

En esta fase el objetivo de try-catch es practicar la captura y tratamiento básico de errores; todavía no se exige un sistema completo de validaciones.

PARTE VIII. Lista de cotejo antes de entregar

Verificación	Listo
El proyecto compila sin errores.	☐
Las clases propuestas existen.	☐
Los atributos tienen tipos adecuados.	☐
Los constructores inicializan objetos.	☐
Se crean objetos con new.	☐
Los objetos realizan llamadas a métodos.	☐
Se utiliza al menos un método con parámetros o retorno, cuando aplique.	☐
Se utiliza BufferedReader.	☐
Se aplica try-catch en una entrada susceptible de error.	☐
Hay comentarios en procesos importantes.	☐

PARTE X. Entrega del Avance N.º 1

- Guía completada o documento equivalente con el análisis y diseño preliminar.
- Proyecto de IntelliJ IDEA con las clases y código desarrollados hasta esta fase.
- Primera prueba funcional donde se creen objetos y se utilicen los métodos implementados.

Revisión oral

Cualquier integrante podrá ser seleccionado para responder preguntas como:

- ¿Por qué identificaron esta clase?
- ¿Qué representa este atributo y por qué ese tipo de dato?
- ¿Qué modificador de acceso utilizaron y por qué?
- ¿Cuál es el constructor y qué inicializa?
- ¿Dónde se instancia este objeto?
- ¿Qué ocurre al utilizar new?
- ¿Qué objeto llama a este método?
- ¿Qué parámetros recibe?
- ¿Retorna algún valor?
- ¿Qué error controlan mediante try-catch?

Criterios de revisión

Criterio	Logrado	Observaciones
Análisis del problema	☐	
Identificación y responsabilidad de clases	☐	
Atributos y tipos de datos	☐	
Constructores y objetos	☐	
Métodos y llamadas	☐	
BufferedReader	☐	
try-catch	☐	
Organización, comentarios y comprensión	☐	

Observaciones de la profesora:

Fecha de revisión: _____ Firma/Iniciales: _____